

A Comparative Analysis of WebAssembly and Docker Containers for Microservice Architectures in Kubernetes

Abdulaziz Musaev

Born on 22nd November, 1999 in Denau, Uzbekistan

Master's Thesis in Distributed Systems Engineering

16 February 2026

Advisor

Dr. Michael Steininger

Referees

Prof. Dr. Christof Fetzer

Dr.-Ing. André Martin

Erklärung

Ich versichere, dass ich alle bewertungsrelevanten Beiträge dieser Arbeit ohne Hilfe anderer Personen und Werkzeuge erbracht habe oder diese Beiträge klar und unmissverständlich im Text gekennzeichnet habe (etwa durch Zitate). Diese Arbeit wurde in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt. Sollte ich mehrere Kopien dieser Arbeit zur Bewertung abgeben, so ist deren Inhalt identisch.

Declaration

I declare that I have achieved all grading-relevant contributions to this work without the help of other persons or tools, or that I have clearly and unmistakably identified such contributions in the text (e.g., by citations). This work has not been submitted in the same or a similar form to any other examination body. Should I submit multiple copies of this work for grading, their content is identical.

(Abdulaziz Musaev)

Dresden, 16 February 2026

Abstract

about 1/2 page:

1. Motivation (Why do we care?)
2. Problem statement (What problem are we trying to solve?)
3. Approach (How did we go about it)
4. Results (What's the answer?)
5. Conclusion (What are the implications of the answer?)

The abstract is a miniature version of the thesis. It should be treated as an entirely separate document. Do not assume that a reader who has access to an abstract will also have access to the thesis. Do not assume that a reader who reads the thesis has read the abstract.

Contents

Abstract	i
1 Introduction	1
1.1 Context	1
1.2 Problem Statement	1
1.3 Research Questions	1
1.4 Research Methodology	1
1.5 Thesis Contributions	1
1.6 Plagiarism Declaration	2
1.7 Thesis Structure	2
2 Background	3
2.1 Cloud Computing and Microservices	3
2.1.1 Cloud Computing Paradigms	3
2.1.2 Microservice Architecture	3
2.2 Containerization Technologies	3
2.2.1 Docker and OCI Containers	3
2.2.2 Kubernetes	4
2.3 WebAssembly (Wasm)	4
2.3.1 Origin and Overview	4
2.3.2 Technical Details	4
2.3.3 Phase 1: Frontend and Browser Adoption	4
2.3.4 Phase 2: Server-Side WebAssembly	4
2.3.5 WASI (WebAssembly System Interface)	5
2.4 Container Orchestration	5
2.4.1 Container Runtimes	5
2.5 Continuum Framework	5
3 State of the Art and Industry Landscape	6
3.1 Theoretical Foundations of Server-Side WebAssembly	6
3.1.1 The Wasm Security Model vs. Container Isolation	6

3.1.2	Performance Characteristics in Academia	6
3.2	The WebAssembly Component Model	6
3.3	Industry Adoption and Ecosystem	7
3.3.1	Edge Computing Providers	7
3.3.2	Application Frameworks	7
3.4	WebAssembly in Kubernetes	7
3.4.1	Evolution of Integrations	7
3.4.2	Modern Operators	7
4	Research Methodology	8
4.1	Research Strategy: Comparative Analysis	8
4.2	Selection of Technologies	8
4.2.1	The Baseline: Docker + Golang	8
4.2.2	The Challenger: WebAssembly + Rust	8
4.2.3	Orchestration: Kubernetes with RuntimeClass	8
4.3	Definition of Metrics	8
5	Implementation of the Microservice Prototype	10
5.1	System Architecture	10
5.2	Baseline Implementation (Golang/Docker)	10
5.2.1	Service Design	10
5.2.2	Container Optimization	10
5.3	Experimental Implementation (Rust/Wasm)	10
5.3.1	Service Design	10
5.3.2	Compilation and Packaging	10
5.4	Kubernetes Cluster Configuration	11
5.4.1	Runtime Class Setup	11
5.4.2	Ingress and Networking	11
6	Evaluation	12
6.1	Benchmark Environment	12
6.2	Results: Startup Performance	12
6.3	Results: Resource Efficiency	12
6.4	Results: Runtime Performance	12
6.5	Qualitative Analysis: Developer Experience	12
7	Discussion and Future Work	13
7.1	The Trade-off: Maturity vs. Efficiency	13
7.2	Security Implications	13
7.3	Migration Recommendations	13

8 Conclusion	14
A Reproducibility	15
A.1 Abstract	15
A.2 Artifact check-list (meta-information)	15
A.3 Description	15
A.3.1 How to access	15
A.3.2 Hardware dependencies	15
A.3.3 Software dependencies	15
A.4 Installation	15
A.5 Experiment workflow	15
A.6 Evaluation and expected results	15
A.7 Notes	15

Chapter 1

Introduction

1.1 Context

The modern cloud landscape is dominated by microservices and containerization. However, as the demand for edge computing and high-density serverless functions grows, the overhead of traditional containers becomes a limiting factor. This section introduces the shift toward lightweight execution environments.

1.2 Problem Statement

Standard OCI containers (like Docker) introduce significant overhead in terms of memory footprint and cold-start latency. This thesis addresses the inefficiency of using full OS-level virtualization for small, transient microservices.

1.3 Research Questions

- **RQ1:** How does the memory footprint of WebAssembly modules compare to traditional Docker containers in a Kubernetes environment?
- **RQ2:** Can WebAssembly runtimes offer superior startup performance compared to standard container runtimes?

1.4 Research Methodology

1.5 Thesis Contributions

1.6 Plagiarism Declaration

1.7 Thesis Structure

Chapter 2

Background

This chapter provides the foundational knowledge required to understand the comparative analysis presented in this thesis. It moves from general cloud concepts to specific container technologies, and finally to a detailed technical overview of WebAssembly.

2.1 Cloud Computing and Microservices

2.1.1 Cloud Computing Paradigms

A brief overview of Cloud Computing, focusing on the evolution from IaaS to FaaS (Function-as-a-Service), highlighting the requirement for rapid elasticity and resource efficiency.

2.1.2 Microservice Architecture

Defining Microservices as an architectural style that structures an application as a collection of loosely coupled services. This section discusses the benefits (scalability, agility) and the challenges (orchestration complexity) that necessitate tools like Kubernetes.

2.2 Containerization Technologies

2.2.1 Docker and OCI Containers

An explanation of Docker, the Open Container Initiative (OCI) standards, and how Linux namespaces and cgroups provide isolation. It establishes the "baseline" against which WebAssembly is compared.

2.2.2 Kubernetes

An introduction to Kubernetes as the de-facto container orchestration platform. Key concepts covered include the API server, Kubelet, Pods, and the Container Runtime Interface (CRI).

2.3 WebAssembly (Wasm)

2.3.1 Origin and Overview

WebAssembly was originally announced in 2015 as a portable, size- and load-time-efficient format suitable for compilation to the web. It was designed to enable high-performance applications on web pages.

2.3.2 Technical Details

- **Binary Format:** Wasm uses a compact binary format that is fast to decode.
- **Stack Machine:** It is defined as a virtual stack machine, unrelated to the physical machine architecture.
- **Linear Memory:** Security is enforced through a linear memory model that prevents the code from accessing memory outside its sandbox.
- **Isolation:** The host environment is completely isolated from the module unless imports are explicitly defined.

2.3.3 Phase 1: Frontend and Browser Adoption

Initially, Wasm was used to port C/C++ applications (like game engines, video editors, and image processors) to the browser, allowing them to run at near-native speed alongside JavaScript.

2.3.4 Phase 2: Server-Side WebAssembly

The "Second Phase" of Wasm involves moving it out of the browser. This section explores why Wasm is ideal for the server side:

- **Security:** Default sandboxing makes it safer for multi-tenant environments.
- **Portability:** "Write once, run anywhere" (creating true universal binaries).
- **Performance:** Near-instant startup times compared to booting a full container OS.

2.3.5 WASI (WebAssembly System Interface)

A critical standard that allows Wasm modules to access system resources (files, network, clocks) in a standardized, secure way, enabling server-side execution.

2.4 Container Orchestration

2.4.1 Container Runtimes

Differentiation between high-level runtimes (Containerd, CRI-O) and low-level runtimes (runc, crun, gVisor).

2.5 Continuum Framework

Chapter 3

State of the Art and Industry Landscape

This chapter reviews the theoretical foundations of server-side WebAssembly and surveys the current industrial adoption of Wasm as a microservice runtime. It contrasts academic findings on isolation and performance with real-world platforms provided by major tech vendors.

3.1 Theoretical Foundations of Server-Side WebAssembly

3.1.1 The Wasm Security Model vs. Container Isolation

An analysis of the "capability-based" security model of WebAssembly (WASI) compared to the Linux namespace and cgroup isolation used by Docker. Discussion includes memory safety, control-flow integrity, and the reduction of the attack surface.

3.1.2 Performance Characteristics in Academia

A review of academic benchmarks comparing Wasm execution speeds (JIT vs. AOT compilation) against native code and interpreted languages (like Golang). Focuses on "Cold Start" latency and memory density.

3.2 The WebAssembly Component Model

Introduction to the Wasm Component Model and WIT (Wasm Interface Type), which solves the "Nano-process" communication problem, allowing microservices to com-

pose without network overhead—a theoretical advantage over traditional HTTP-based microservices.

3.3 Industry Adoption and Ecosystem

3.3.1 Edge Computing Providers

- **Cloudflare Workers:** Utilization of V8 Isolates for near-instant startup.
- **Akamai Linode:** Integration of Wasm for edge compute, highlighting their recent acquisition of Fermyon technologies.
- **Fastly Compute:** Use of the Lucet (now Wasmtime) runtime for sub-millisecond instantiation.

3.3.2 Application Frameworks

- **Fermyon Spin:** A developer-centric framework for building event-driven microservices.
- **WasmCloud:** An actor-model-based platform focusing on distributed systems and "location transparency".
- **WasmEdge:** A CNCF sandbox project optimized for AI/ML workloads and extension of cloud-native ecosystems.

3.4 WebAssembly in Kubernetes

3.4.1 Evolution of Integrations

From the deprecated *Krustlet* project to modern *containerd* shims (runwasi).

3.4.2 Modern Operators

- **SpinKube:** The de-facto standard for running Spin apps in K8s.
- **WasmCloud Operator:** Bridging Kubernetes orchestration with the WasmCloud lattice.
- **KWasm:** An operator that automates the installation of Wasm shims on K8s nodes.

Chapter 4

Research Methodology

4.1 Research Strategy: Comparative Analysis

This thesis employs a quantitative comparative analysis (A/B testing) to evaluate the suitability of Wasm/Rust against Docker/Golang.

4.2 Selection of Technologies

4.2.1 The Baseline: Docker + Golang

Justification for choosing Golang (FastAPI/Flask) as the baseline: it represents the industry standard for rapid development but suffers from high memory usage and slow startup (the "Garbage Collection" penalty).

4.2.2 The Challenger: WebAssembly + Rust

Justification for Rust: it provides the smallest Wasm binary sizes and lacks a garbage collector, maximizing the potential benefits of the Wasm runtime.

4.2.3 Orchestration: Kubernetes with RuntimeClass

Description of the hybrid cluster setup where both artifacts will run side-by-side.

4.3 Definition of Metrics

- **Startup Latency (Cold Start):** Time from 'kubectl scale' to the first HTTP 200 OK response.

-
- **Memory Footprint:** RSS (Resident Set Size) memory usage at idle and under load.
 - **Artifact Size:** Comparison of the OCI image size (Golang slim layers) vs. the Wasm module size.
 - **Throughput Latency:** Requests Per Second (RPS) and P99 latency under sustained load (using tools like K6 or Hey).
 - **Maintainability (Qualitative):** Assessment of the "Inner Development Loop," debugging difficulty, and CI/CD complexity.

Chapter 5

Implementation of the Microservice Prototype

5.1 System Architecture

Description of the "Reference Microservice" logic (e.g., a stateless service that performs image processing or cryptographic calculations to stress CPU and Memory).

5.2 Baseline Implementation (Golang/Docker)

5.2.1 Service Design

Implementation details using Golang (FastAPI).

5.2.2 Container Optimization

Efforts to minimize the Docker image (e.g., using Alpine Linux or Distroless images) to ensure a fair comparison.

5.3 Experimental Implementation (Rust/Wasm)

5.3.1 Service Design

Re-implementation of the logic in Rust using the 'spin-sdk' or 'wasmcloud-actor' SDK.

5.3.2 Compilation and Packaging

Compiling to 'wasm32-wasi' and packaging as an OCI artifact.

5.4 Kubernetes Cluster Configuration

5.4.1 Runtime Class Setup

Configuring 'containerd' to use 'runwasi' or 'spin-shim' for specific workloads.

5.4.2 Ingress and Networking

Configuring Traefik or Nginx to route traffic to both the Wasm and Docker services.

Chapter 6

Evaluation

6.1 Benchmark Environment

Hardware specifications of the Kubernetes cluster (local Minikube vs. Managed Cloud K8s) and the load testing tools used.

6.2 Results: Startup Performance

Presentation of data showing the "Cold Start" gap between the Golang VM initialization vs. Wasm JIT instantiation.

6.3 Results: Resource Efficiency

Analysis of memory density. (Expected Result: Wasm modules using <5MB vs Golang containers using >50MB).

6.4 Results: Runtime Performance

Comparison of raw CPU throughput. Discussion on whether the lack of Garbage Collection in Rust/Wasm offsets the overhead of the Wasm runtime sandbox compared to Native Golang.

6.5 Qualitative Analysis: Developer Experience

Documentation of the challenges faced during the Rust migration and K8s integration (e.g., debugging Wasm modules, ecosystem maturity).

Chapter 7

Discussion and Future Work

7.1 The Trade-off: Maturity vs. Efficiency

Discussion on the "Ecosystem Gap." While Wasm is more efficient, Golang/Docker has vastly superior library support and tooling availability.

7.2 Security Implications

How the "deny-by-default" security posture of Wasm changes the cluster security model compared to root-access risks in Docker.

7.3 Migration Recommendations

Guidelines for companies (like Akamai or Cloudflare customers) on when to switch to Wasm (e.g., high-scale serverless functions) versus staying with Docker (stateful monoliths).

Chapter 8

Conclusion

Summary of findings confirming whether Wasm/Rust serves as a viable alternative for the specific use case of high-density microservices.

Appendix A

Reproducibility

A.1 Abstract

A.2 Artifact check-list (meta-information)

A.3 Description

A.3.1 How to access

A.3.2 Hardware dependencies

A.3.3 Software dependencies

A.4 Installation

A.5 Experiment workflow

A.6 Evaluation and expected results

A.7 Notes

List of Abbreviations

List of Figures

List of Tables
